KUBERNETES

Study Note

Kubernetes (K8s) is an open-source container orchestration platform. It automates the deployment, scaling, and management of containerized applications.

Key Benefits of Kubernetes over Docker :

- **Multi-host orchestration**:

Kubernetes manages containers across many machines (nodes), not just one. This allows better use of resources and scalability.

- **Auto-scaling with HPA**:

It can automatically increase or decrease the number of running Pods based on resource usage like CPU or memory.

HPA (Horizontal Pod Autoscaler)

HPA automatically scales the number of Pods based on CPU/memory usage.

- **Self-healing:**

If a container crashes or a node fails, Kubernetes restarts or reschedules the affected Pods automatically.

- **Rolling updates and rollbacks**:

Kubernetes updates apps gradually without downtime. If the new version fails, it can quickly roll back to the previous version.

- **Load balancing built-in**:

Kubernetes distributes traffic to the right Pods using Services, so apps stay available and responsive.

- **Works with CI/CD and cloud providers**:

Kubernetes integrates easily with DevOps tools (like GitHub Actions) and runs on all major cloud platforms.

Differ. Nodes and Pods

In Kubernetes, a **Node** is a machine—either physical or virtual—that provides the computing power to run applications.

A **Pod** is the smallest unit in Kubernetes that runs your application. It wraps around one or more containers (usually just one) and gives them everything they need to run—like shared storage, network, and settings. All containers in a Pod share the same IP address and can talk to each other easily.

Feature	Node	Pod
Definition	A physical or virtual machine in the Kubernetes cluster	The smallest deployable unit in Kubernetes
Purpose	Runs one or more Pods	Runs one or more containers
Contains	OS, kubelet, container runtime (e.g., Docker/Containerd)	Application containers, storage, and network resources
Scope	Cluster-level resource	Application-level abstraction
Managed By	Kubernetes control plane	Controller (e.g., Deployment, ReplicaSet)
Networking	Has its own IP address	Each Pod gets its own IP address inside the Node
Lifecycle	Managed as a compute resource	Created and destroyed frequently as per app lifecycle
Analogy	A room in a hotel	A bed inside the room

Kubernetes Architecture

🧠 Control Plane (The brain of the cluster)

The Control Plane is responsible for the overall management of the Kubernetes cluster. It makes decisions like where to run applications, keeps track of cluster state, and handles changes.

Key Components:

1. **kube-apiserver**
    - It's the front-end of the Kubernetes control plane.
    - All requests (e.g., from kubectl) go through the API server.

- It validates and processes all cluster changes.

2. **etcd**
    - A distributed key-value store used as the cluster's **database**.
    - Stores all cluster state: configurations, secrets, pod details, etc.
    - A backup of etcd is essential for disaster recovery.
3. **kube-scheduler**
    - Decides **which Node** should run a new Pod.
    - Chooses based on resource availability, labels, taints, etc.
4. **kube-controller-manager**
    - Ensures the actual cluster state matches the desired state.
    - Examples: making sure there are always 3 replicas of a pod.
    - Includes various controllers like Node Controller, Replication Controller, etc.
5. **cloud-controller-manager**
    - Integrates Kubernetes with cloud providers (like AWS, GCP, Azure).
    - Manages cloud-specific resources: load balancers, volumes, etc.

⚙️ Data Plane (The workers of the cluster)

The Data Plane is made up of **Nodes** that run your actual application containers inside **Pods**. These components interact with the Control Plane to receive instructions and execute them.

Key Components:

1. **kubelet**
    - An agent running on each Node.
    - It communicates with the control plane and ensures the containers are running in the Pods as expected.

- It reports Pod status and health back to the control plane.

2. **kube-proxy**
    - Manages network rules on each Node.
    - It handles routing traffic to the right Pod using services.
    - Enables communication between Pods and external users.
3. **Container Runtime**
    - The engine that runs containers (e.g., Docker, containerd, CRI-O).
    - Kubernetes uses this to actually start and stop containers.

Running a Pod on Minikube (with Docker Desktop as Backend)

🧰 Prerequisites:

Make sure you have these installed:

- ✅ **kubectl** (Kubernetes CLI)
- ✅ **Minikube**
- ✅ **Docker Desktop** running in the background (Minikube will use it as the container runtime)

Step-by-Step Setup Process

Step 1: Start Minikube with Docker driver

Start a single-node Kubernetes cluster (Minikube):

```
minikube start --driver=docker
```

This:

- Launches a Kubernetes cluster locally.
- Uses Docker (from Docker Desktop) as the container runtime.

Step 2: Write the Pod YAML file

Create a file named pod.yml with this content:

E.g.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

YAML Breakdown: What each line means

Line	Explanation
apiVersion: v1	Defines the version of the Kubernetes API to use. For Pods, we use v1.
kind: Pod	You're creating a Pod resource. Pods are the smallest deployable unit in Kubernetes.
metadata:	Metadata provides info about the Pod.
name: nginx	The name you’re giving to your Pod (nginx). Must be unique within the namespace.
spec:	Defines the specification of the Pod — what's inside it.

Line	Explanation
containers:	A list of containers to run in this Pod. Even though one is typical, multiple containers can be in one Pod.
- name: nginx	The name of the container (used for logs, debugging, etc.)
image: nginx:1.14.2	The container image to use. This will pull the official nginx version 1.14.2 from Docker Hub.
ports:	Tells Kubernetes which ports the container exposes internally.
- containerPort: 80	Port 80 is the default port for Nginx to listen on.

Step 3: Create the Pod

kubectl create -f pod.yml

If successful, you'll see:

pod/nginx created

Step 4: Check Pod status and IP

kubectl get pods -o wide

Example output:

```
NAME    READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
nginx   1/1     Running   0          20s   10.244.0.23   minikube   <none>           <none>
```

Note the IP address (e.g., 10.244.0.23) — it's internal to the cluster.

Step 5: SSH into Minikube VM

You **can't access Pod IPs directly** from your Windows host, but you **can from inside Minikube**:

minikube ssh

Then inside Minikube, test Nginx:

curl 10.244.0.4

You should get the default Nginx HTML page in response.

✅ This confirms your Pod is running and reachable inside the cluster.

Step 6: To delete the Pod:

```
kubectl delete pod nginx
```

🧠 Summary of Concepts

Concept	Description
Minikube	Local Kubernetes cluster tool for testing
Docker Desktop	Used as the VM & container runtime by Minikube
Pod	Smallest Kubernetes unit containing containers
kubectl create	Creates resources from a file (here, the Pod)
-o wide	Shows extra details like IP and Node
minikube ssh	Let's you run commands **inside** the cluster
curl	Test connectivity to Pod IP from inside Minikube

📘 kubectl describe pod <pod-name>

This command provides **detailed information** about a specific Pod in Kubernetes. It can be user for troubleshot the pods.

🔍 What it shows:

- **Metadata** (name, namespace, labels, annotations)
- **Node** the Pod is running on

- **Pod IP** and **status**
- **Container details** (image, ports, env vars)
- **Events** (scheduling, pulling image, starting container, errors if any)

Deployment and ReplicaSets & Controllers

🔷 1. Container vs Pod vs Deployment

Concept	Description
Container	A single instance of a running application (like a Nginx server) — runs in isolation.
Pod	A wrapper around one or more containers. It is the **smallest deployable unit** in Kubernetes. Pods share storage/network and run in the same environment.
Deployment	A Kubernetes resource that manages **Pods**. It ensures desired replicas, rolling updates, and self-healing. It uses a **ReplicaSet** behind the scenes.

🔷 2. Difference Between Pods and Deployments

Pods	Deployments
A Pod is a one-time, single unit.	A Deployment **manages** Pods.
Created manually or via YAML.	Creates Pods automatically via ReplicaSet.
Not self-healing. If it crashes, it's gone.	Self-healing. Lost Pods are recreated.
Good for testing or one-off runs.	Good for production, scalable apps.

🔷 3. What Is a ReplicaSet?

- A **ReplicaSet** is a Kubernetes controller that ensures a **specified number of pod replicas** are running at any given time.
- A **Deployment** creates and manages ReplicaSets.
- ReplicaSet watches for any **missing or crashed pods** and brings them back automatically.

🚑 **Self-healing** behaviour = thanks to the ReplicaSet.

deployment.yaml

```
apiVersion: apps/v1              # API version for Deployment
kind: Deployment                 # Type of Kubernetes object
metadata:
  name: nginx-deployment         # Name of the deployment
  labels:
    app: nginx                   # Labels to identify and group
spec:
  replicas: 3                    # Desired number of Pods
  selector:
    matchLabels:
      app: nginx                 # Tells ReplicaSet which Pods to manage
  template:                      # Blueprint for Pods
    metadata:
      labels:
        app: nginx               # Labels applied to each Pod
    spec:
      containers:
      - name: nginx              # Container name
        image: nginx:1.14.2      # Docker image to use
        ports:
        - containerPort: 80      # Container listens on port 80
```

We can find the syntax from Kubernetes deployment documents in web for this example nginx image. Also, there is number examples are available at GitHub repositories also.

🔷 5. kubectl apply vs kubectl create

kubectl create	kubectl apply
Creates a resource once.	Creates or updates a resource.
Fails if resource already exists.	Merges your YAML with existing config.
Good for one-time setups.	Good for continuous deployments (like GitOps).

🔄 You used to apply so you could edit the replica count and re-apply without deleting the resource.

6. Autohealing by ReplicaSets

A ReplicaSet in Kubernetes is a powerful *controller* that ensures a specified number of pod replicas are always running at any given time. Its primary job is to maintain the desired state of your application by continuously monitoring and managing the number of running pods.

If a pod fails, gets deleted, or crashes, the ReplicaSet immediately detects the change and automatically creates a new pod to replace it. This self-healing capability is what makes Kubernetes highly reliable and resilient. Unlike individual pods—which are ephemeral and will not be recreated if deleted manually—ReplicaSets guarantee availability by watching over them.

It uses a *selector* to match the labels of the pods it controls and ensures that the number of pods matching those labels always equals the number specified in the replicas field.

For example, if you declare replicas: 3 and one pod is removed or fails, the ReplicaSet controller will automatically launch a new identical pod to restore the count to 3. In this way, ReplicaSet plays a crucial role in Kubernetes auto-healing, ensuring your application is continuously available without manual intervention.

Testing ReplicaSet Behaviour

did:

1. Deployed 3 replicas.
2. Run:

kubectl get pods -w

This watches Pods in real time.

3. Deleted a Pod manually:

kubectl delete pod <pod-name>

After deletion:

- The ReplicaSet detects that only 2 Pods are running (but 3 are desired).
- It automatically **creates a new pod** to maintain the replica count.

📌 This is how Kubernetes maintains **desired state**. You'll see a new pod name appear — that's auto-healing in action!

🔍 View the ReplicaSet directly

kubectl get rs

You'll see the ReplicaSet with:

- DESIRED = 3
- CURRENT = 3
- READY = 3

This shows the ReplicaSet is actively ensuring the correct state.

StatefulSet and DaemonSet in Kubernetes

What is a StatefulSet in Kubernetes?

A StatefulSet is used to manage stateful applications, where each Pod needs to be uniquely identifiable, stable, and persistent.

📜 Key Features:

1. Stable Network Identity: Each Pod gets a persistent hostname like pod-0, pod-1, etc.
2. Persistent Storage: Each Pod gets its own volume, which stays the same even if the Pod is deleted and recreated.
3. Ordered Deployment & Scaling:
    - Pods are created in order (0, 1, 2…).
    - Pods are deleted in reverse order (2, 1, 0…).
4. Use Case: Databases (e.g., MongoDB, Cassandra), Kafka, etc., where each instance maintains its own data and state.

📦 Example:

If you deploy 3 replicas of a MongoDB cluster using StatefulSet:

- You get: mongo-0, mongo-1, mongo-2
- Each has a unique DNS: mongo-0.mongo-service.default.svc.cluster.local

🔁 What is a DaemonSet in Kubernetes?

A DaemonSet ensures a copy of a Pod runs on every node in the cluster.

📜 Key Features:

1. One Pod per Node: As soon as a node joins, the DaemonSet ensures a Pod runs on it.
2. Used for background tasks or node-specific agents.
3. Automatic updates: If a node is added, the Pod is deployed there automatically.

🧰 Common Use Cases:

- Log collection agents (e.g., Fluentd, Filebeat)
- Monitoring agents (e.g., Prometheus Node Exporter)
- Network tools or storage plugins

Feature	StatefulSet	DaemonSet
Purpose	Stateful applications	Node-wide utilities
Pod Uniqueness	Yes (pod-0, pod-1, etc.)	No, same pod spec on every node
Volume	Unique persistent volumes per pod	Optional, often doesn't use volumes
DNS Hostname	Yes (Stable and predictable)	No
Scaling	Manual control of replicas	Automatically matches node count
Example Use Case	Databases, Kafka	Log collectors, Monitoring tools

Kubernetes Services

A **Kubernetes Service** is an **abstract way to expose a group of Pods** (typically running the same application) as a **network service**. Even though Pods are ephemeral (they can die and be replaced), a Service gives them a **stable IP** and **DNS name**, allowing other components or users to communicate with them reliably.

🔑 **In simple words**:
A Service gives a fixed **name (DNS)** and **IP address** to a group of Pods, so you can reach your application **reliably**, even if the Pods themselves are temporary.

Importance of Services in Production

In real-world **production environments**, your application:

- Needs to scale by adding more replicas (Pods)
- Must handle **external traffic** from users and APIs
- Should maintain **high availability** with **automatic load balancing**
- Should **automatically discover** backends even when Pods change

This is where Kubernetes Services are **critical components** — they ensure stable communication, traffic routing, and service discovery in dynamic environments.

Replicas and Load Balancing

When you create **multiple replicas** of a Pod using Deployments, Kubernetes schedules those Pods across nodes. But:

- Users shouldn't have to know about each Pod's IP
- Traffic should be distributed evenly among replicas

That’s where Services come in — they **load balance** incoming requests across the available replicas of a Pod.

Service and Load Balancing in Kubernetes

A Kubernetes Service acts like a **built-in load balancer** that:

- Watches Pods using **label selectors**
- Routes traffic evenly to healthy Pods
- Ensures that **users** or **internal components** don’t need to track changing Pod IPs

You don’t need to assign a separate IP to each user project. Instead, you can create a single **LoadBalancer Service**, and Kubernetes will take care of the rest.

Application Exposure (Internal & External)

Kubernetes Services can expose your app:

- **Internally** (to other apps inside the cluster)
- **Externally** (to the outside world via public IP or DNS)

This is crucial for building APIs, websites, or microservices that must be accessible from **browsers**, **external users**, or **third-party systems**.

Service Discovery

Service Discovery in Kubernetes is the mechanism by which **one part of your application (like a frontend)** can **find and communicate with another part (like a backend)** — without needing to hardcode IP addresses.

✅ It answers the question:

"How can one Pod find and talk to another Pod in a scalable, reliable way?"

Why Service Discovery is Needed

In Kubernetes:

- **Pods are ephemeral** (they can die, restart, and get a new IP).

- **Multiple replicas** of the same app might exist for scaling and load balancing.
- IP addresses **change dynamically**.

So:

- Hardcoding IPs of Pods is **impossible and unreliable**.
- Kubernetes Services act as **stable endpoints**.
- Service Discovery lets clients dynamically find backend Pods through Services.

Kubernetes uses two main components to achieve service discovery:

1. DNS-based Service Discovery

Kubernetes runs a DNS server (like CoreDNS) inside the cluster. Every Service gets a **DNS name;** Any Pod can use this name to reach the Service.

2. Environment Variable-based Discovery

When a Pod is created, Kubernetes injects environment variables for each Service into the Pod.

However, this is less flexible, because:

- It's static — added only at Pod creation.
- New Services won't be available to already running Pods.

So, DNS is preferred and used in real-world applications.

Labels and Selectors in Kubernetes Service Discovery

What Are Labels?

Labels are key-value pairs attached to Kubernetes objects like Pods, Services, Deployments, etc.

They are used to organize, identify, and select subsets of objects.

Syntax Example (Pod with labels):

```
labels:
  app: nginx
  environment: production
  tier: frontend
```

These labels are arbitrary, meaning you can define your own keys and values, like:

Why Are Labels Important?

Kubernetes objects (like Services, Deployments, ReplicaSets) use **selectors** to match against these labels to decide **which Pods to target or manage**.

Think of it like:

🔎 "Find me all Pods where app=nginx" — that's what a selector does.

What Are Selectors?

Selectors are queries that match labels. They are used by:

- Services (to route traffic to Pods)
- Deployments (to manage ReplicaSets and Pods)
- ReplicaSets (to track and maintain Pods)

Example:

```
selector:
  app: nginx
```

This selector matches any Pod with the label app=nginx.

How Labels & Selectors Work Together for Service Discovery

Imagine you have 3 Pods running your app, all labeled with:

```
labels:
  app: myapp
```

Now, you create a Service like this:

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 8080
```

🔁 **What Happens Behind the Scenes?**

1. The Service uses the selector app: myapp.

2. Kubernetes finds all Pods with this label.
3. It automatically **creates and updates an internal list of endpoints** (IP addresses of Pods).
4. When someone accesses the Service (via DNS or IP), traffic is **load balanced to one of the matched Pods**.

✅ This is **dynamic**. If a new Pod with the label app: myapp is added, the Service automatically includes it.

❌ If a Pod without that label is running, it will not be part of the Service.

Summary

Term	Definition
Label	Key-value metadata to identify Kubernetes objects
Selector	A query that matches labels to select specific objects
Service	Uses selectors to find Pods to route traffic to
Service Discovery	The ability for Services to automatically find and connect to Pods

Key Benefits of Kubernetes Service Discovery

Feature	Description
Dynamic routing	Automatically discovers new Pods via labels & selectors
DNS-based communication	Services are accessible using predictable DNS names
Load balancing	Traffic is balanced between healthy Pods
Decoupling	Frontend and backend don’t need to know about each other's IPs
Resilience	If Pods crash or restart, traffic reroutes to available ones

Kubernetes Services Types

Kubernetes provides **3 major service types**, each designed for **specific networking needs**:

Service Type	Accessible From	Typical Use Case
ClusterIP	Inside the cluster only	Internal communication between Pods
NodePort	Outside via Node IP:Port	Basic external access for dev/test
LoadBalancer	Public IP via cloud LB	Production-grade external access

1. ClusterIP (Default)

What is it?

- The default type of Service.
- Exposes the application on a virtual IP (ClusterIP) that's accessible only inside the cluster.

Use Case:

- Internal communication between:
  - Frontend ↔ Backend
  - App ↔ Database
- Microservices communicating inside Kubernetes.

How it Works:

- You define the selector to point to Pods (e.g., app=backend).
- Kubernetes assigns a stable ClusterIP like 10.96.0.5.
- Other Pods can reach it using:
  - DNS: http://my-service
  - Or IP directly: http://10.96.0.5

Security:

- Cannot be accessed from outside the cluster.
- Perfect for secure internal networks.

1. ClusterIP (Default)

What is it?

- The default type of Service.
- Exposes the application on a virtual IP (ClusterIP) that's accessible only inside the cluster.

Use Case:

- Internal communication between:
  - Frontend ↔ Backend
  - App ↔ Database
- Microservices communicating inside Kubernetes.

How it Works:

- You define the selector to point to Pods (e.g., app=backend).
- Kubernetes assigns a stable ClusterIP like 10.96.0.5.
- Other Pods can reach it using:
  - DNS: http://my-service
  - Or IP directly: http://10.96.0.5

Security:

- Cannot be accessed from outside the cluster.
- Perfect for secure internal networks.

2. NodePort

What is it?

- Makes the Service **accessible outside the cluster** using:

<NodeIP>:<NodePort>

- Kubernetes allocates a **static port (30000–32767)** on **each Node**.

Example:

Let's say:

type: NodePort

nodePort: 30007

Then:

curl http://<minikube-ip>:30007

Notes:

- Not suitable for production.
- Port range is limited.
- You must manage traffic routing to different apps manually.

3. LoadBalancer Service

A LoadBalancer type service in Kubernetes exposes your application to the internet via a public IP address and routes incoming traffic through a cloud provider's external load balancer.

It builds on top of the NodePort and ClusterIP services but adds an external Load Balancer (LB) in front of them.

✅ Best for production environments where real users need stable and scalable external access to your app.

How it Works (Behind the Scenes)

Here's what happens when you create a LoadBalancer service in a cloud-supported Kubernetes cluster (like EKS, GKE, AKS):

1. You define a service with type: LoadBalancer.
2. Kubernetes automatically:
    - Creates a ClusterIP for internal routing.
    - Opens a NodePort on each node.
    - Tells the cloud provider (via cloud controller manager) to:
        - Provision a Load Balancer (e.g., AWS ELB, GCP LB).
        - Assign a public IP address.
        - Forward traffic from LB → NodePort → Pods.
3. The external LB will then:
    - Receive traffic on public IP.
    - Use health checks to monitor Pod health.
    - Distribute requests to healthy Pods.

Why LoadBalancer is Important

- **Stable External Access**: Assigns a consistent public IP (or domain).
- **Smart Load Balancing**: Distributes traffic to Pods.
- **Health Checks**: Automatically checks backend Pod health.
- **Scalability**: Works with Horizontal Pod Autoscaler (HPA).
- **Cloud-Native Integration**: Uses native cloud features under the hood (e.g., AWS ELB, Azure LB).

Things to Remember

🔍 Concern	📌 Detail
Cloud provider required	Works only in managed Kubernetes like GKE, EKS, AKS
Extra cost	Cloud Load Balancers are billable
Slow provisioning	Initial LB creation might take 1–2 minutes
One LB per service	Every LoadBalancer service gets a new LB
DNS not included	You must point your domain (e.g., myapp.com) to the LB's IP manually

Kubernetes Ingress

1. What is Ingress?

Ingress is a Kubernetes **API object** that allows **external traffic (HTTP/HTTPS)** to reach **internal Kubernetes services** in a **smart and efficient** way.
Instead of assigning each service a separate external IP or port, you define **Ingress rules** to handle:

- **Path-based routing** (/app1 goes to Service A, /app2 to Service B)
- **Host-based routing** (app1.example.com, app2.example.com)
- **TLS termination** (HTTPS support)
- **Authentication, redirects**, etc.

📌 **Ingress = Intelligent entry point** to your Kubernetes applications.

2. Why Ingress?

Without Ingress, exposing multiple services means:

- Using a **NodePort** or **LoadBalancer** per service
- Higher **resource usage**, **cost**, and **complexity**

Problems Without Ingress:

Problem	Example
Too Many IPs	Each service gets a LoadBalancer IP (costly in cloud)
Hard to Manage	Every service has its own port/IP rules
Scattered TLS	TLS setup per service, hard to maintain
DevOps Overhead	Managing 10+ apps = 10+ exposure methods

✅ **What Ingress Solves:**

- Uses a **single IP/domain** for **many services**
- **Centralized routing and rules**
- **Simpler, more scalable**, and **cost-effective**

Ingress acts like a **gatekeeper + traffic director** at the cluster edge.

3. Load Balancer IP vs Ingress

Feature	LoadBalancer	Ingress
Scope	One service per IP	Multiple services with a single IP
Routing	No routing logic	Smart path/host-based routing
TLS/HTTPS	Needs setup per service	Centralized TLS termination
Cloud Cost	High (IP per service)	Efficient (one IP for all)
Use Case	Simple exposure	Complex, multi-service apps

Real-world Analogy:

- **LoadBalancer**: Like having a separate road for each shop.
- **Ingress**: Like a **single mall entrance** with signboards guiding you to each shop inside.

4. What is Ingress Controller?

An **Ingress Controller** is a **Kubernetes component** (usually a Pod) that:

- **Listens** to Ingress resources (rules)
- **Configures a reverse proxy** (like NGINX, Traefik)
- **Handles real traffic** and routes it correctly

⚠️ Just creating an Ingress resource is not enough! You **must install and run an Ingress Controller** for the Ingress to actually work.

🔄 **Working:**

1. You define an Ingress rule (e.g., /api → service backend)
2. The Ingress Controller sees it
3. It updates its internal reverse proxy
4. Incoming request to /api is routed to the backend service

5. Types of Ingress Controllers

There are multiple **Ingress Controllers**, each using a different proxy engine or designed for specific environments:

Ingress Controller	Description	Best For
NGINX Ingress Controller	Most popular; works in any environment	Minikube, on-prem, cloud
Traefik	Lightweight, dynamic config updates, modern dashboard	Dev environments, dynamic workloads
HAProxy Ingress	High performance, enterprise-grade	High traffic workloads
AWS ALB Ingress Controller	Integrates directly with AWS ALB	AWS EKS

For Minikube or local testing, NGINX Ingress Controller is highly recommended and easy to set up.

Kubernetes ConfigMaps & Secrets

1. What is a Kubernetes ConfigMap?

A ConfigMap is a Kubernetes API object used to store non-confidential configuration data in key-value format.

Purpose:

- To decouple application code from configuration.
- Helps you change configuration without rebuilding container images.

Examples of ConfigMap content:

- App config settings
- URLs, filenames
- Command-line flags
- App environment (ENV=production)
- File contents like .env or app.properties

💡 Think of ConfigMap as an external config file for your container apps in Kubernetes.

2. What is a Kubernetes Secret?

A Secret is a Kubernetes object similar to ConfigMap but designed to store sensitive data, such as:

- Passwords
- API keys
- Certificates
- SSH keys
- OAuth tokens

💡 Key Feature:

- Data in Secrets is Base64 encoded
- Kubernetes ensures they are stored and transmitted securely
- Kubernetes supports integrating Secrets with volume mounts or environment variables

🔒 Use Secrets for any confidential config that shouldn't be visible in plaintext.

3. Difference between ConfigMap and Secret

Feature	ConfigMap	Secret
Data Type	Non-sensitive	Sensitive (confidential)
Encoding	Plaintext	Base64-encoded
Security	No encryption by default	Designed for secure storage
Use Case	App settings, environment data	Passwords, keys, tokens
Access	ENV variables or volumes	ENV variables or volumes
Storage	etcd	etcd (can be encrypted)

📌 Rule of thumb: If the data can be safely shared or logged, use ConfigMap. If not, use Secret.

4. Using ConfigMap as ENV Vars inside a Container App

✅Step-by-Step:

1. Create ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  APP_MODE: production
  LOG_LEVEL: info
```

2. Use it in Pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: myapp
    image: myapp:1.0
    envFrom:
```

```
  - configMapRef:
      name: my-config
```

🧪 All keys inside the ConfigMap will be automatically injected as environment variables.

5. Using ConfigMap as Volume Mount inside a Container App

✅ **Step-by-Step:**

1. **Create ConfigMap** with file-style data:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: config-vol
data:
  app.properties: |
    APP_MODE=production
    LOG_LEVEL=info
```

2. Use it in Pod Volume:

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-app
spec:
  containers:
  - name: app-container
    image: myapp:1.0
    volumeMounts:
    - name: config-volume
      mountPath: /etc/config
  volumes:
  - name: config-volume
    configMap:
      name: config-vol
```

📂 Kubernetes will create a file per key inside the /etc/config directory.
For example: /etc/config/app.properties contains the config text.

✅ Advantages of Using Volume Mounts over ENV Vars for ConfigMaps

Feature	ENV Variables	Volume Mounts
Data Size Limit	Limited (small env vars)	Large files supported
Format	Key-value only	Complex structured config (YAML, JSON, .properties)
Reload Capability	Requires Pod restart	Can reload dynamically if app supports
Ease of Access	Simple for basic settings	Best for config files
Access	ENV variables or volumes	ENV variables or volumes
Storage	etcd	etcd (can be encrypted)

💡 Use **ENV vars** for simple, short settings.
Use **Volume mounts** for structured config files, secrets, or large data.

How Kubernetes Secrets Work Internally

🔄 Secret Lifecycle:

1. **You define** a Secret object (YAML or CLI).
2. Kubernetes **stores** it in etcd, optionally encrypted.
3. You **mount** it into Pods using env or volume.
4. Applications read these values at runtime.
5. Kubernetes **does not log or expose** the secret content.

🧠 Example Secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data:
  password: cGFzc3dvcmQ=  # base64 of "password"
```

Mount as ENV in Pod:

```
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: my-secret
      key: password
```

Mount as Volume:

```
volumeMounts:
- name: secret-volume
  mountPath: /etc/secret
volumes:
- name: secret-volume
  secret:
    secretName: my-secret
```

Summary

Feature	ConfigMap	Secret
Purpose	App configuration	Confidential data
Encoding	Plaintext	Base64
Access	ENV / Volume	ENV / Volume
Use When	Config you can log	Credentials or tokens
Security	Not encrypted	Designed for security
Storage	etcd	etcd (can be encrypted)

Kubernetes provides a clean separation of config from code using ConfigMaps and Secrets, enabling secure, scalable, and manageable application deployments.

Kubernetes RBAC (Role-Based Access Control)

What is RBAC in Kubernetes?

RBAC (Role-Based Access Control) is a method of **regulating access to resources** in Kubernetes based on the roles of individual **users** or **service accounts**.

📌 **Why RBAC?**

- To control **who can do what** inside your Kubernetes cluster.
- Helps enforce **least privilege access**: users get only the permissions they need.
- RBAC is built into Kubernetes and uses standard Kubernetes resources to define permissions.

Kubernetes Role vs ClusterRole

What is a Role in Kubernetes?

A Role in Kubernetes is a namespace-scoped object that defines a set of permissions (rules) to access Kubernetes resources within a specific namespace.

🔎 Purpose of Role:

It helps you control access within a single namespace.

For example:

- A developer may only be allowed to read pods in the dev namespace.
- A monitoring app may be allowed to list and watch resources in a specific namespace.

✅ Example: A Role that allows reading Pods in a namespace

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
rules:
```

```
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

Explanation:

- namespace: dev: The role is limited to the dev namespace.
- resources: ["pods"]: Only pods are included.
- verbs: What actions are allowed → get (read), list (see all), watch (observe changes).

📌 What is a ClusterRole in Kubernetes?

A ClusterRole is similar to a Role, but it is not limited to a namespace. It is cluster-scoped and can be used to:

1. Give access to cluster-wide resources (like nodes, persistentvolumes, namespaces).
2. Apply the same access across all namespaces.
3. Assign admin-level or read-only permissions cluster-wide.

✅ Example: A ClusterRole that allows reading Nodes

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
```

Explanation:

- This ClusterRole allows access to nodes, which are not namespace-scoped resources.
- This permission applies to entire cluster, not just one namespace.

Users' vs Service Accounts

What Are Users in Kubernetes?

In Kubernetes, users represent real humans (like developers, admins, testers) who interact with the cluster externally using tools like:

- kubectl
- CI/CD pipelines
- APIs

Kubernetes does NOT have a built-in user management system. It relies on external systems (IDPs like LDAP, OIDC) to manage and authenticate users.

What Are Service Accounts?

ServiceAccounts are Kubernetes-native identities used by applications (Pods) running inside the cluster to interact with the Kubernetes API.

🔶 Every Pod is automatically assigned a default ServiceAccount (unless you specify one).

🔶 You can create your own ServiceAccount and assign it to a Pod if needed.

Feature	Users	ServiceAccounts
Represents	Real people (external users)	Apps/Pods running inside the cluster
Managed by	External IDP (OIDC, LDAP)	Kubernetes itself
Used by	kubectl, CI tools	Containers, controllers
Authenticated by	Certificates, OIDC tokens	ServiceAccount tokens
Namespace-bound	✕ No	☑ Yes (created in a namespace)
Example	dev-user, admin-user	default, backend-app-serviceaccount

How to Create Users in Kubernetes?

Kubernetes **doesn't manage users natively**. Instead, we simulate or authenticate users via:

✅ Static TLS Certificates (for dev/testing)

Create a user by:

1. Generating a key pair (private key + certificate).
2. Signing with the cluster's CA.
3. Adding user config to ~/.kube/config.

But this is **not scalable** in production.

✅ **Production: Use Identity Providers (IDP)**

For real-world authentication, Kubernetes relies on external **Identity Providers**:

Identity Provider Type	Examples
LDAP / Active Directory	Enterprise user databases
OIDC (OpenID Connect)	Google, GitHub, Okta, Azure AD
SAML	Keycloak, OneLogin, Auth0

Kubernetes API Server is configured to **delegate** user login to these providers.

How RBAC Works with Users & ServiceAccounts

🔁 **RBAC = Role-Based Access Control**

Kubernetes uses **RBAC** to **authorize** (allow/deny) access after authentication.

RBAC Flow:

1. User or Pod makes a request (kubectl, API)
2. Kubernetes authenticates identity
3. RBAC checks:
   - Does this identity have permission to do this action?
   - via Role/ClusterRole and Binding
4. If yes → access granted
   If no → access denied

Real-Life Example: RBAC Flow

Scenario:

- Alice (developer) wants to list pods in the dev namespace.
- Admin has created a **Role** allowing pod read access in dev.
- Admin binds it to Alice using a **RoleBinding**.

RBAC Objects:

```
# Role (dev namespace)
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
```

```
metadata:
  namespace: dev
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]

# RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-alice
  namespace: dev
subjects:
- kind: User
  name: alice@example.com
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

✅ When Alice runs: kubectl get pods -n dev

➜ Authenticated via OIDC

➜ RBAC checks binding

✅ Access granted to list pods

🤖 Example: ServiceAccount & RBAC Flow

If a backend Pod uses a backend-serviceaccount to access ConfigMaps, we do the same:

1. Create a **Role** with read access to ConfigMaps.
2. Create a **RoleBinding** for the **ServiceAccount**.

```
# Role
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: app-space
```

```
  name: config-reader
rules:
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get", "list"]
```

```
# RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: config-reader-binding
  namespace: app-space
subjects:
- kind: ServiceAccount
  name: backend-serviceaccount
roleRef:
  kind: Role
  name: config-reader
  apiGroup: rbac.authorization.k8s.io
```

✅ The app running in the Pod now **securely reads config data** without hardcoding credentials.

🧠 Key Takeaways

- **Users** = external humans, managed by external systems like OIDC.
- **ServiceAccounts** = internal app identities, managed by Kubernetes.
- RBAC links **Roles** (what can be done) to **Subjects** (users or service accounts) via **Bindings**.
- Use **IDPs** for production-grade user management.
- ServiceAccounts are **scoped to a namespace**, useful for least-privilege design.